Projet Semestriel

CO	Description	Ce TP
CO1	Understand OS fundamentals, Linux command-line environment, and operating system service interfaces	✓
CO2	Understand process scheduling, memory management, and file system organization algorithms	✓
CO3	Manage Linux users and permissions, implement shell scripts, and apply text processing tools	✓

Introduction

Le projet semestriel (TP11) est un travail **intégrateur** qui mobilise les trois résultats d'apprentissage du module (CO1, CO2, CO3) : les fondamentaux et la ligne de commande Linux, les sous-systèmes du noyau (processus, mémoire, fichiers) et l'administration/automatisation. Il vous demande de concevoir et de développer, en groupe, un outil Linux complet que vous **défendrez oralement** en fin de semestre.

Cinq projets vous sont proposés, de difficultés variées : l'audit et l'analyse de journaux (`watchdog`) ainsi que le monitoring avec automatisation (`automon`) couvrent l'administration et l'observation du système ; les mini-shells (`opsh` en Python, `ocsh` en C) vous font reconstruire un interpréteur de commandes ; enfin, le noyau simulé (`esinos`) reproduit l'ordonnancement, la mémoire et le système de fichiers d'un SE.

Chaque projet vous amènera à découvrir de **nouveaux concepts** et de **nouvelles commandes** qui ne figurent pas au programme : c'est une **excellente occasion d'apprendre** par vous-même.

Difficulté et note maximale

Les projets varient en complexité : la **note maximale** atteignable dépend du projet choisi. Une soumission **parfaite** d'un projet plus difficile dépasse toujours une soumission parfaite d'un projet plus facile.

Projet	Sujet	Difficulté	Note max
Projet 1	`watchdog` — Audit & forensics	Intermediate	16/20
Projet 2	`automon` — Monitoring & automation	Intermediate	17/20
Projet 3	`opsh` — Mini-shell (en Python)	Challenging	18/20
Projet 4	`ocsh` — Mini-shell (en C)	Challenging	19/20
Projet 5	`esinos` — Mini-noyau simulé	Advanced	20/20

Règles de constitution des groupes

* Groupes de **4 étudiants maximum**.
* **Les membres d'un groupe doivent appartenir au même groupe de TP.** Le mélange des sections n'est pas autorisé.
* Choix du projet via un **formulaire Microsoft** envoyé par l'enseignant.

Format de la soutenance

* La soutenance a lieu pendant votre **séance de TP habituelle**.
* La discussion dure **15 minutes maximum** par groupe (démo, revue de code et questions).
* 4 cartes tirées au hasard : **2 CODE** et **2 CONCEPT**.
* Les 2 étudiants « CODE » expliquent le code ; les 2 « CONCEPT » expliquent les concepts.
* L'attribution est aléatoire : chaque membre doit maîtriser les deux volets.
* **Absence le jour de la soutenance = 0**, sans rattrapage.

Livrables

Soumettez un dépôt **GitHub public** contenant :

* `code/` — code source et scripts bien commentés
* `report.pdf` — 2 à 4 pages : approche, difficultés, résultats et liens avec le cours
* `README.md` — description de votre projet, accompagnée d'instructions d'exécution détaillées
* `demo.mp4` — une vidéo de démonstration, **caméra ouverte**, présentant le travail et expliquant le code

Projet 1 — `watchdog` : Audit & forensics

Difficulté : Intermediate — Note max : 16/20

Objectif

Construire un outil Bash unique qui combine l'analyse de journaux et l'audit des permissions pour produire un rapport de sécurité unifié.

Données fournies

`access.log` et `auth.log` (dossier `data/logwatch/`) + `accounts.txt` (mini `passwd` + permissions, dossier `data/auditx/`).

Ce que vous devez construire

* Un script `watchdog.sh` avec deux sous-commandes : `logs` et `audit`
* `watchdog.sh logs -f access.log -top-ips -404 -failed-logins` : classement des IP, comptage des 404 et des échecs d'authentification (`grep` / `sed` / `awk` / `sort` / `uniq`)
* `watchdog.sh audit -f accounts.txt -weak-perms -suid -umask` : permissions effectives (octal), fichiers « world-writable », binaires SUID/SGID
* `watchdog.sh report` : rapport de sécurité unifié (logs + audit)
* Gestion des arguments, messages d'aide et codes de retour

Démo en soutenance

1. Lancer `logs` sur les journaux fournis : montrer le top IP, les 404 et les échecs d'authentification.
2. Lancer `audit` : montrer les permissions faibles et expliquer le calcul octal sur un exemple.
3. Générer et ouvrir le rapport unifié.

Concepts de base (à couvrir)

* stdin/stdout/stderr ; pipes et redirections comme IPC ; expressions régulières (BRE vs ERE)
* `grep`, `sed`, `awk`, `sort`, `uniq` ; pourquoi composer de petits outils
* UID/GID ; permissions `rwx` sur fichiers vs répertoires ; notation octale ; `umask`
* Bits SUID/SGID et sticky ; sudo / élévation de privilèges

Projet 2 — `automon` : Monitoring & automation

Difficulté : Intermediate — Note max : 17/20

Objectif

Construire un outil Bash qui combine le monitoring temps réel du système et un mini pipeline d'intégration continue.

Données fournies

Un projet d'exemple (programme C `hello.c` + script de test `run_tests.sh` + `build.conf`) est fourni dans `data/autobuild/sample/`.

Ce que vous devez construire

* `automon.sh monitor` : tableau de bord temps réel (façon `top`) construit depuis `/proc` : `/proc/loadavg`, `/proc/stat`, `/proc/meminfo`, `/proc/[0-9]*/stat`, `/proc/[0-9]*/cmdline` ; options `-refresh` et `-top`
* `automon.sh build` / `test` / `report` / `archive` : mini pipeline CI piloté par `build.conf`, journalisation pass/fail, archivage daté (`tar`)
* Affichage de la ligne `cron` pour planifier l'exécution
* Sortie propre via `trap` et gestion des codes de retour `$?`

Démo en soutenance

1. `monitor` : observer CPU / charge / mémoire se rafraîchir, trier par mémoire, quitter proprement.
2. `build` puis `test` sur le projet d'exemple : montrer le résultat pass/fail.
3. `report` puis `archive` (archive datée) ; afficher la ligne `cron`.

Concepts de base (à couvrir)

* `/proc` : pseudo-système de fichiers fourni par le noyau ; états des processus ; load average ; temps CPU ; RAM vs swap ; arbre des processus (PPID)
* Chaque étape du pipeline est un processus ; codes de retour ; redirection et journalisation
* Signaux ; `cron` ; archivage avec `tar`

Projet 3 — `opsh` : Mini-shell en Python

Difficulté : Challenging — Note max : 18/20

Objectif

Construire un interpréteur de commandes fonctionnel en Python : `opsh` lit une commande, la lance et supporte les mécanismes réels de Bash. Vous implémentez vos propres **commandes renommées** ainsi que les redirections, les pipes et les signaux.

Vous implémentez ces commandes vous-même : les **builtins** dans le shell, les **alias** vers les commandes Bash.

Builtins		Alias	
commande bash	**commande opsh**	**commande bash**	**commande opsh**
`cd`	`go`	`ls`	`list`
`exit`	`quit`	`cat`	`show`
`pwd`	`path`	`cp`	`copy`
`echo`	`say`	`mv`	`move`
`help`	`info`	`rm`	`del`
`history`	`past`	`mkdir`	`newdir`
`pipe ( | )`	`pipe ( -> )`	`touch`	`newfile`

Le shell doit aussi fournir le builtin `osinfo`, qui affiche `Systèmes d'Exploitation -- UIR/ESIN`, et l'opérateur de pipe `->` (au lieu de `|`).

Ce que vous devez construire

* Boucle REPL : prompt `opsh>` → lecture → découpage par `->` → exécution via `os.fork()` / `subprocess`
* Builtins et alias de la table ci-dessus ; redirections `>` `»` `<`
* Pipes `->` (câblage des descripteurs) ; arrière-plan `&` et signaux `SIGINT` / `SIGCHLD`
* Variables et expansion : `$VAR`, `${VAR}`, `export`, `$?`, `$!`
* Substitution `$(...)` / backticks ; `.opshrc` ; historique et complétion (`readline`)
* Mode script `opsh script.opsh` (`if` / `for` / `while`) ; **optionnel** : job control `jobs` `fg` `bg`

Démo en soutenance

1. `opsh> list` · `go ..` puis `path` · `say hello > f.txt` puis `show f.txt`
2. `opsh> show f.txt -> wc -l` (pipe `->`) · `sleep 10 &` puis `ps` · `osinfo`

Concepts de base (à couvrir)

* Programme vs processus ; fork/exec/wait ; états ; zombie / orphelin
* Pourquoi `go` / `cd` doit être un builtin (non forkable) ; redirection (descripteurs 0/1/2) ; pipes comme IPC ; signaux
* Mode utilisateur vs noyau ; PATH / environnement

Projet 4 — `ocsh` : Mini-shell en C

Difficulté : Challenging — Note max : 19/20

Objectif

Construire un interpréteur de commandes en C, en appelant directement les **appels système** du noyau : création de processus, redirections, pipes et signaux. Vous implémentez vos propres **commandes renommées** ainsi que le cœur de Bash.

Vous implémentez ces commandes vous-même : les **builtins** dans le shell, les **alias** vers les commandes Bash.

Builtins		Alias	
commande bash	**commande ocsh**	**commande bash**	**commande ocsh**
`cd`	`go`	`ls`	`list`
`exit`	`quit`	`cat`	`show`
`pwd`	`path`	`cp`	`copy`
`echo`	`say`	`mv`	`move`
`help`	`info`	`rm`	`del`
`history`	`past`	`mkdir`	`newdir`
`pipe ( | )`	`pipe ( -> )`	`touch`	`newfile`

Le shell doit aussi fournir le builtin `osinfo`, qui affiche `Systèmes d'Exploitation -- UIR/ESIN`, et l'opérateur de pipe `->` (au lieu de `|`).

Ce que vous devez construire

* Boucle REPL : lecture → découpage par `->` → `fork()` → `execvp()` → `wait()`
* Builtins et alias de la table ci-dessus ; redirections `>` `»` `<` via `dup2`
* Pipes `->` (câblage des descripteurs avec `pipe`) ; arrière-plan `&` et signaux `SIGINT` / `SIGCHLD`
* Variables et expansion : `$VAR`, `${VAR}`, `export`, `$?`, `$!`
* Substitution `$(...)` / backticks ; `.ocshrc` ; historique et complétion
* Mode script `ocsh script.ocsh` (`if` / `for` / `while`) ; **optionnel** : job control `jobs` `fg` `bg`

Démo en soutenance

1. `ocsh> list` · `go ..` puis `path` · `say hello > f.txt` puis `show f.txt`
2. `ocsh> show f.txt -> wc -l` (pipe `->`) · `sleep 10 &` puis `ps` · `osinfo`

Concepts de base (à couvrir)

* Programme vs processus ; fork/exec/wait ; pourquoi `exec` remplace l'image du processus
* Pourquoi `go` / `cd` doit être un builtin (non forkable) ; zombie / orphelin ; états
* Pipe comme IPC ; signaux ; frontière appels système (user / kernel)

Projet 5 — `esinos` : Noyau de système d'exploitation simulé

Difficulté : Advanced — Note max : 20/20

Objectif

Une simulation en espace utilisateur d'un système d'exploitation avec trois sous-systèmes : un **ordonnanceur**, un **gestionnaire mémoire** et un **mini système de fichiers**.

Ce que vous devez construire

- **Ordonnanceur** : FCFS, SJF, RR, Priority ; lecture d'un fichier de tâches ; diagramme de Gantt + temps d'attente/séjour moyens
- **Gestionnaire mémoire** : pagination, traduction logique → physique, remplacement de pages FIFO / LRU / Optimal, TLB et temps d'accès effectif (EAT)
- **Mini système de fichiers** : allocation à base d'inodes sur un fichier-disque, création / lecture / suppression
- Un menu intégrant les trois sous-systèmes

Démo en soutenance

1. Lancer le menu (`1 Scheduler · 2 Memory · 3 FS`).
2. Ordonnanceur : afficher les Gantt FCFS / SJF / RR.
3. Mémoire : montrer l'éviction FIFO pas à pas, puis comparer avec LRU.
4. Système de fichiers : créer / lire / supprimer et montrer la table d'inodes.

Concepts de base (à couvrir)

- FCFS / SJF / RR / Priority ; diagramme de Gantt
- Temps d'attente / de séjour ; préemption
- Page vs cadre ; défaut de page
- FIFO / LRU / Optimal ; TLB et EAT ; mémoire virtuelle
- Inode et allocation de blocs